In this example, the line attr_accessor :price creates both the getter method that allows you to write puts book.price and the setter method that allows you to write book.price = book.price * 0.75.

Attributes Are Just Methods Without Arguments

These attribute-accessing methods do not have to be just simple wrappers around an object's instance variables. For example, you may want to access the price as an exact number of cents, rather than as a floating-point number of dollars.

tut_classes/stock_stats/book_in_stock_7.rb

```
class BookInStock
  attr_reader :isbn
  attr_accessor :price

  def initialize(isbn, price)
    @isbn = isbn
    @price = Float(price)
  end

  def price_in_cents
    (price * 100).round
  end
  # ...
end

book = BookInStock.new("isbn1", 33.80)
puts "Price          = #{book.price}"
puts "Price in cents = #{book.price_in_cents}"
```

produces:

```
Price          = 33.8
Price in cents = 3380
```

We multiply the floating-point price by 100 to get the price in cents and then use the round method to convert it to an integer. Why? Because floating-point numbers don't always have an exact internal representation. When we multiply 33.8 by 100, we get 3379.99999999999954525265. The Integer method would truncate this to 3379. Calling round ensures we get the best integer representation. This is a good example of why you want to use BigDecimal, not Float, in financial calculations, see Chapter 26, Library Reference: Core Data Types, on page 495 for more on BigDecimal.

We can take this even further and create a writing method parallel to the reader method, mapping the value to the instance variable internally:

tut_classes/stock_stats/book_in_stock_8.rb

```
class BookInStock
  attr_reader :isbn
  attr_accessor :price

  def initialize(isbn, price)
    @isbn = isbn
    @price = Float(price)
  end

  def price_in_cents
    (price * 100).round
  end
```